

Functional & Non-Functional Requirements

Functional Requirements :

- o User Accounts: Citizens can directly view availability of beds and blood without login. Citizens also have the option to sign up for further updates in the future. (Not mandatory) Hospital/Blood bank admins can sign up and log in to update their data.
- o Hospital Beds: Show citizens which hospitals have beds available (ICU, Oxygen, General). Allow filtering by location and type of bed. Clearly mark beds as *Available*, *Occupied*, or *Under Maintenance*.
- o Blood Units: Let users search for blood groups in nearby blood banks. Show how much blood is available and when it expires. Allow citizens to place a request for blood.
- o Emergency Requests: Citizens can send an emergency request (for a bed, blood). The system should alert the hospital or service provider.
- o Admin Dashboard: Admins should be able to add or update dummy data (beds, blood) in the prototype. Every action (like updating bed status or recording blood unit changes) should be logged with date and time for accountability.
- o Search & Filters: Provide a search bar and filters (by location, type, availability) so users can find help faster.
- o Future Ready: The first version will use simulated data entered by admins, but it is designed to connect with real hospital and blood bank APIs in the future.

Non-Functional Requirements :

- o Fast Performance: The app should load results quickly (within 2 seconds for normal searches). It should work smoothly even if many people use it at the same time.
- o Always Available: The web application should work 24/7 with very little downtime. Regular backups should be taken so data is not lost.
- o Easy to Grow (Scalable): New hospitals, blood banks, should be added easily. The system should handle large numbers of users during big emergencies (like floods, pandemics).
- o Secure: User data like passwords and personal info should be safe and encrypted. Only verified admins should be able to change important data. Admin activity logs should never be editable or deleted.
- o Easy to Maintain: The system should be modular and well-documented so future developers can update it easily. The database should be designed in a way that makes adding new features simple.

o User Friendly: The design should work well on mobile phones since most users will use it in emergencies. The interface should be simple and clear so that users don't waste time figuring out how to use it. Information like dates, phone numbers, and locations should be shown in a standard, consistent format.